中山大学计算机学院

人工智能实验报告

课程名称：Artificial Intelligence

教学班级	超算班	专业	信息与计算科学
学号	21307261	姓名	王健阳

一，实验题目

朴素贝叶斯文本分类

二，实验内容

算法原理

根据训练集里的众多文本数据，对每个单词对应的情感进行一个概率预测。而对于测试集里面的每一个句子，要预测其情感，首先将其分解为一个个单词，再在训练集中逐行遍历目标句子每个单词的概率进行累乘，并在最后乘上训练集该行句子处于该感情的概率。最后逐行求和得到总的概率预测值。

概率计算

为了预测文本 $X_3 = (x_3, x_4)$ 的情感 e_i，我们需要估计：

$$p(e_i|X_3) \propto \sum_{j=1}^{2} p(x_3|e_i, d_j)p(x_4|e_i, d_j)p(d_j, e_i)$$

$$\begin{aligned} p(joy|X_3) \propto\ & p(x_3|joy, d_1)p(x_4|joy, d_1)p(d_1, joy) \\ & +p(x_3|joy, d_2)p(x_4|joy, d_2)p(d_2, joy) \\ & = 0.17 \times 0.17 \times 0.9 + 0.25 \times 0 \times 0.3 = \mathbf{0.02601} \end{aligned}$$

$$\begin{aligned} p(sad|X_3) \propto\ & p(x_3|sad, d_1)p(x_4|sad, d_1)p(d_1, sad) \\ & +p(x_3|sad, d_2)p(x_4|sad, d_2)p(d_2, sad) \\ & = 0.17 \times 0.17 \times 0.1 + 0.25 \times 0 \times 0.7 = \mathbf{0.00289} \end{aligned}$$

伪代码

```
//导入并存储训练集数据
train_data = train_data_init()
test_data = test_data_init()

//对于测试集里的每一个句子
for sentence in test_data:
    p_list = []
    for emotion in emotion_list:
```

```
        //求p(ei|Xi)
        p_ei_xi = 0
        for word in sentence:
            for _sentence in train_data:
                temp = 1
                for _word in _sentence:
                    temp *= p(_word, emotion)
                temp *= p(_sentence, emotion)
                p_ei_xi += temp
    //预测结果为概率最大值对应的情感
    emotion_predict = max_emotion(p_list)
```

关键代码展示

> tools.py

```
import copy
import random
from sklearn.feature_extraction.text import TfidfVectorizer
import numpy as np
# 训练集情感分布
# anger: 6.6%, disgust: 2.6%, fear: 16%, joy: 36.2%, sad: 20.2%, surprise: 18.4%


emotion_list = ['anger', 'disgust', 'fear', 'joy', 'sad', 'surprise']
train_data_length = 20


# 词语集频率归一化
def regulate(result):
    for row_index in range(len(result)):
        total = sum(result[row_index])
        for i in range(len(result[row_index])):
            result[row_index][i] /= total
    return result


# 拉普拉斯变换
def Lapras(matrix: list, sentence_length_list: list):
    for index, row in enumerate(matrix):
        # 得到每一行的最小公倍数
        word_num = sentence_length_list[index]
        extra = 0.35
        for i in range(len(row)):
            row[i] = (row[i] * word_num + extra) / (word_num + extra *
len(matrix[0]))


# 更加规范合理地存放训练数据
class TrainData:
```

```
    sparse_matrix = None  # 稀疏矩阵简便存储
    emotion_data = None  # 存储每一行的情感概率分布
    train_data_length = None
    name_list = None  # 矩阵列标题

    # 传入稀疏矩阵的列标题，完整稀疏矩阵，对应的行情感列表
    # 关键是对于测试集相应数据的提取
    def __init__(self, name_list, sparse_matrix, emotion_listed,
sentence_length_list):
        Lapras(sparse_matrix, sentence_length_list)
        self.name_list = name_list
        self.sparse_matrix = sparse_matrix
        self.pof_emo_init(emotion_listed)
        self.train_data_length = len(sparse_matrix)
        for i in self.emotion_data[:10]:
            # print(i)
            pass

    # 对于每一行
    def pof_emo_init(self, emotion_listed):
        self.emotion_data = []
        for cur_emo in emotion_listed:
            temp_dict = {}
            for emo in emotion_list:
                if emo == cur_emo:
                    temp_dict[emo] = 0.9
                else:
                    temp_dict[emo] = 2.6%


def Predict(sentence: str, train_data: TrainData):
    global emotion_list
    word_list = sentence.split(' ')
    p_list = []
    # 对于每一种情感
    for emo in emotion_list:
        emo_prob = 0
        # 对训练集里面的每一行求和
        for row in range(train_data.train_data_length):
            prob_per_row = 1
            # 对句子里的每一个单词相对于此行的概率求积
            for word in word_list:
                if word in train_data.name_list:
                    index = list(train_data.name_list).index(word)
                    atom_prob = train_data.sparse_matrix[row][index]
                else:
                    # 对于不在训练集中的词，当其不存在，故置为1
                    atom_prob = 1
                if atom_prob:
                    prob_per_row *= atom_prob
            prob_per_row *= train_data.emotion_data[row][emo]
            emo_prob += prob_per_row
        p_list.append(emo_prob)
    # print(p_list)
```

```
    # end of initializing p_list
    answer = emotion_list[p_list.index(max(p_list))]
    return answer
```

main.py

```
from tools import *


# 分别读取训练集以及测试集，进行文本提取
def data_init():
    file_train = open('Classification/train.txt', 'r')
    file_test = open('Classification/test.txt', 'r')
    file_train_txt = file_train.read().split('\n')[1:-1]
    file_test_txt = file_test.read().split('\n')[1:-1]

    # 分离训练集中的句子和情感，分别存储在
    train_text = []
    train_emotion = []
    sentence_length_list = []
    for _item in file_train_txt:
        _temp = _item.split(' ')
        train_text.append(' '.join(_temp[3:]))
        train_emotion.append(_temp[2])
        sentence_length_list.append(len(_temp) - 3)
    _tv = TfidfVectorizer(use_idf=False)
    _tv_fit = _tv.fit_transform(train_text)
    _ft_name = _tv.get_feature_names_out()

    # 获得训练集的完整稀疏矩阵
    sparse_matrix = regulate(_tv_fit.toarray())
    data = TrainData(_ft_name, sparse_matrix, train_emotion, sentence_length_list)
    return file_test_txt, data


if __name__ == '__main__':
    # 主函数开始
    test_text, train_data = data_init()

    denominator = len(test_text)
    # denominator = 100
    numerator = 0
    # 对于测试集的每一个数据，进行结果的比对
    cnt = 0
    for item in test_text[:]:
        temp = item.split(' ')
        emotion_predict = Predict(' '.join(temp[3:]), train_data)
        if emotion_predict == temp[2]:
            numerator += 1
        cnt += 1
        if cnt % 10 == 0:
            print(emotion_predict, end=' ' if cnt % 200 != 0 else '\n')
```

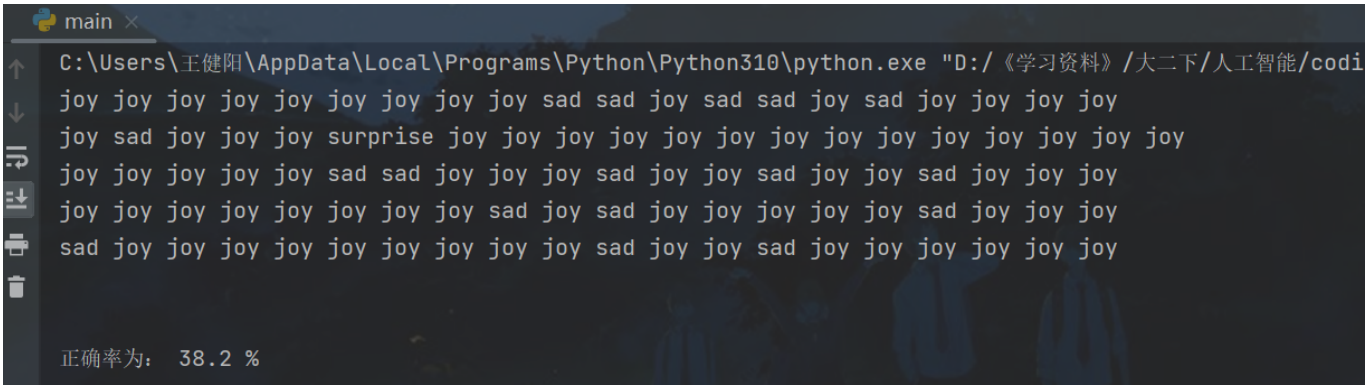
```
        # print(emotion_predict, end=' ' if cnt % 30 != 0 else '\n')
    print('\n\n正确率为: ', 100 * round(numerator / denominator, 5), '%')


# 记录拉普拉斯变化赋予不同值的正确率变化，最终稳定在38.2
'''
0.1     0.2     0.3     0.325     0.33     0.34     0.345     0.35     0.375
0.4
37.5    37.6    37.8    38.0      38.1     38.1     38.2      38.2     37.8
37.7

'''
```

创新点 & 优化

暂无

三，实验结果及分析

> 结果展示



> 分析

由于测试集过于庞大，此处只打印了10的倍数对应的预测结果。最后的预测值取到了局部最大值38.2，估计是此数据集的最好结果了。限于训练集，很多单词无法给出预测结果。对于这些不在训练集的单词，赋予多少权重（赋0等于直接摆烂）也意义不大，因为最后的概率筛选是在同一个句子里选择概率最大的情感作为预测值。权重改变无非拉大抑或是缩小差距，但无法逆转结果。故此处统一设为1。除去贝叶斯估计，其他可以调参的地方也寥寥无几，故将38.2作为最终的结果。（直接返回joy就有36.2的正确率，感觉低于这个准确率的代码都毫无意义。。。）

四，思考题

思考：在前面的文本分类算法中，如果测试文本中的单词没有在训练文本中出现会造成什么结果？

此问题在上述分析时已做解答，会大大降低预测准确率。（取极限情况，测试集单词训练集中完全没有，就完全无法预测）

五，参考资料

1.实验python基础pdf

2.CSDN